

Multi-class Classification - Reuters Dataset

Dataset by Reuters in 1986 as a set of short newswires and their topics. (<https://keras.io/api/datasets/reuters/> (<https://keras.io/api/datasets/reuters/>)) Previously we looked at predicting a binary-classification problem; however, the same techniques can be used to predict many different classes. In the case of the Reuters dataset, we have 46 topics which can be predicted.

Much of what we're covering here are detailed in Chollet (2021) Section 3.4 Getting Started with Neural Networks: Classification and Regression - 4.2 Classifying newswires: A multiclass classification example, if you wish to also read along.

Requirements

As previously, if we encounter a `ModuleNotFoundError` error, we should install the appropriate packages. You should already have all relevant packages from last week; however, you can uncomment the magic-commands below if you need to reinstall these.

```
In [1]: 1 %pip install keras
        2 %pip install tensorflow
        3 %pip install numpy
        4 %pip install scikit-learn
        5 %pip install pandas
```

We can get rid of those pesky info / warning messages which tensorflow outputs and clogs up our output with the following. Make sure to do this prior to importing tensorflow.

In detail:

- 0 = all messages are logged (default behavior)
- 1 = INFO messages are not printed
- 2 = INFO and WARNING messages are not printed
- 3 = INFO, WARNING, and ERROR messages are not printed

```
In [2]: 1 import os
        2 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
```

Introduction

```
In [ ]: 1 from keras.datasets import reuters
        2
        3 # Utility function provided by Keras.
        4 # num_words limits the results to the most frequent 10K words in the dataset; these will be our number of features.
        5 # test_split - The function already performs shuffled train-test split for us,
        6     #we can define how big the test-split should be.
        7 # seed - Random seed for shuffling. Delete if you want randomness everytime.
        8
        9 #reuters.load_data(num_words = 10000)
       10 (X_train, y_train), (X_test, y_test) = reuters.load_data(num_words = 10000, test_split = 0.3, seed=42)
```

```
In [4]: 1 print(f"X Train length: {len(X_train)}")
        2 print(f"y Train length: {len(y_train)}")
        3 print(f"X Test length: {len(X_test)}")
        4 print(f"y Test length: {len(y_test)}")
        5
        6 print(f"Total data examples: {len(X_train) + len(X_test)}")
```

```
X Train length: 7859
y Train length: 7859
X Test length: 3369
y Test length: 3369
Total data examples: 11228
```

Let's investigate what the data looks like.

In [5]:

```
1 print(X_train[0])
2
3 # And the Label
4 print(y_train[0]) # -> 3
```

```
[1, 39, 566, 11, 14, 841, 11, 29, 53, 617, 187, 193, 15, 14, 134, 533, 15, 53, 175, 3758, 948, 15, 14, 4313, 279, 15, 3
9, 4074, 11, 14, 3814, 11, 123, 248, 867, 377, 1471, 81, 2, 64, 187, 6164, 2116, 617, 79, 335, 7, 48, 14, 187, 6164, 46
56, 7, 105, 324, 27, 69, 12, 18, 730, 1698, 5, 4, 134, 248, 867, 377, 171, 2, 75, 40, 251, 18, 79, 5, 175, 324, 27, 4,
867, 377, 171, 17, 12]
3
```

Our input data is simply a sequence of word indices. E.g [1, 39, 556, 11] is a sequence of word 1, 39, 556, and 11. This seems cryptic, but each index here links to a vocabulary entry. That is, a mapping from id -> word.

In [6]:

```
1 vocabulary = reuters.get_word_index()
2 vocabulary = {v:k for k,v in vocabulary.items()} # Invert the mapping. Numeric indices as keys, words as values.
3
4 for k,v in vocabulary.items():
5     print(k, v)
```

```
10996 mdbl
16260 fawc
12089 degussa
8803 woods
13796 hanging
20672 localized
20673 sation
20675 chanthaburi
10997 refunding
8804 hermann
20676 passengers
20677 stipulate
8352 heublein
20713 screaming
16261 tcby
185 four
1642 grains
20680 broiler
12090 wooden
1220 wednesday
```

For our first news article (index 0), let's go through each of those word indices and look up what the corresponding word from the article is. This should give us a better idea that the articles are just encoded as numbers linking to a fixed vocabulary. With the problem being numeric, we can now begin to solve *this* problem.

Note: If we were to change the vocabulary mapping, then any trained models would no longer work and we'd need to re-train on the new mapping.

```
In [ ]: 1 for word_idx in X_train[0]:
        2     print( vocabulary.get(word_idx - 3, '?') )
        3     # Indices 0, 1, and 2 are reserved. Therefore we offset by 3. If a word cannot be found, use a '?'.
```

Pre-processing

At the moment our labels are integers. For Categorical Cross-entropy, we will require a column per outcome. E.g If we have 3 classes, then our y label should have 3 features. We call this `one-hot encoding`. Where the appropriate class is set to 1 (hot).

E.g For Class 3 we would have the vector `[0, 0, 1]`; Class 2 `[0, 1, 0]`; and Class 1 `[1, 0, 0]`. Vectors such as `[0, 0, 0]` and `[1, 1, 0]` or `[1, 1, 1]` would be erroneous.

E.g For a 5 class problem we would have a vector `[0, 0, 0, 0, 0]`, then just set the class = 1. E.g Class 3 would be `[0, 0, 1, 0, 0]`.

We will use this same concept to convert our input data - a 10,000 word vocabulary mapping - as well as our target/labels ready for our neural network to understand.

```

In [8]: 1 import numpy as np
2
3 # First create an empty numpy matrix full of 0.
4 # Shape will be the number of rows of data we have, and at each row we will have a 10,000 column vector.
5 x_train_enc = np.zeros( shape=(len(X_train),10000) )
6 print(x_train_enc.shape)
7
8 # E.g 7859, 10000
9 # Each row has a 10K dimension vector stored. Each index of that 10K vector represent a word index.
10
11 for row_number, word_idx_seq in enumerate(X_train):
12     # Whenever we have a word_index, set the appropriate index in the 10K vector to 1
13     # E.g [ 1, 39, 566 ] would set index 1, 39, and 566 of our 10K vector to 1. All else would be 0.
14     x_train_enc[ row_number, word_idx_seq] = 1
15
16 print(x_train_enc[0])
17
18
19 ### Same for the x_test set now. Yes, we could make a function for this.
20
21
22 x_test_enc = np.zeros( shape=(len(X_test),10000) )
23 for row_number, word_idx_seq in enumerate(X_test):
24     x_test_enc[ row_number, word_idx_seq] = 1
25
26 print(x_test_enc[0])
27
(7859, 10000)
[0. 1. 1. ... 0. 0. 0.]
[0. 1. 1. ... 0. 0. 0.]

```

We can use a numpy utility to help us convert the labels. Going from integers -> vectors for each row.

```

In [9]: 1 from keras.utils.np_utils import to_categorical
2 y_train_enc = to_categorical(y_train)
3 y_test_enc = to_categorical(y_test)

```

```
In [10]: 1 print(y_train_enc.shape) # 46 dimensional vector now.
```

```
(7859, 46)
```

Model Definition

Now we have our input data in a format which works for us (one-hot encoding), let's define a model. Previously we defined quite a small Neural Network as our 'problem' was much simpler (wine). Today we're going to explore how changing these impacts our loss metrics, and how we can make some decisions on our architecture based on those.

```
In [11]: 1 import tensorflow as tf
2 from tensorflow import keras
3 from tensorflow.keras import layers
```

Model from last week:

```
model = keras.Sequential()
model.add(layers.InputLayer(input_shape=(11,))) # 11 Columns of input
model.add(layers.Dense(32, activation="relu"))
model.add(layers.Dense(1, activation="sigmoid")) # 0->1 floating
model.summary()
```

A few changes we will make:

- Input shape changing from 11-dimension to 1000-dimension
- Additional hidden layer
- Output number of neurons will be 46 - to represent the number of news topics.
- Change of activation in final layer to `softmax`. Softmax outputs a probability distribution. We want a N-dimensional vector output where we can just take the `argmax` to find the predicted class.

```
In [12]: 1 model = keras.Sequential()
2 model.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3 model.add(layers.Dense(64, activation="relu"))
4 model.add(layers.Dense(64, activation="relu")) # 2 Hidden Layers
5 model.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
6 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	640064
dense_1 (Dense)	(None, 64)	4160
dense_2 (Dense)	(None, 46)	2990

=====
 Total params: 647,214
 Trainable params: 647,214
 Non-trainable params: 0

Previously we specified `optimizer='sgd'`, if we want more control we should create our optimisers explicitly and define their values. This will become important in later weeks. For now we will define a standard SGD optimiser, with default learning rate.

```
from keras.optimizers import SGD

model.compile(...
    optimizer=SGD(learning_rate=0.01)
...)
```

Additionally, as we are **NOT** doing a binary classification problem we must change our loss function to account for the categorical approach we've taken. Instead of `binary_crossentropy` we will be using `categorical_crossentropy`

```
model.compile(
    loss='categorical_crossentropy',
    ...
)
```

```
In [13]: 1 from tensorflow.keras.optimizers import SGD
          2
          3 # Compile the model.
          4 model.compile(
          5     loss='categorical_crossentropy',
          6     optimizer=SGD(learning_rate=0.05), # Higher Learning rate
          7     metrics=['accuracy']
          8 )
```

Training

Here we fit our model using the standard procedure from last week. We stored the training history in an object for plotting later, providing metrics per epoch. We provide training x and y data, as well as a number of epochs.

A new change for this week, is the introduction of the `batch_size` keyword. This allows us to control the mini-batch which is used in the optimiser (SGD). Recall from last week's reading the range from entire-data to single example for calculation of gradients. Mini-batches are the middle ground. By default Keras uses batch sizes of 32. It should be noted that this parameter has diminishing returns. Using a batch size of 64 is not twice as good as 32. For more complex data, we run into memory limitations where we cannot physically store that many data examples in one-go.

Additionally, we're going to provide a validation set for it to calculate validation loss automatically on. We're going to grab 1000 examples from our training set to hold back for this purpose. This is important for us to make quick decisions from the training and validation set differences as to how to tweak our model.

In [14]:

```
1 # Slice notation
2 x_val = x_train_enc[:1000] # Grab from 0 -> 1000
3 y_val = y_train_enc[:1000]
4
5 x_train_enc_rest = x_train_enc[1000:] # From 1000 -> end
6 y_train_enc_rest = y_train_enc[1000:]
7
8 model_training_history = model.fit(
9     x_train_enc_rest,
10    y_train_enc_rest,
11    batch_size = 512,
12    epochs = 120,
13    validation_data = (x_val, y_val)
14 )
```

Epoch 1/120

14/14 [=====] - 1s 16ms/step - loss: 3.6383 - accuracy: 0.3380 - val_loss: 3.2605 - val_accuracy: 0.4490

Epoch 2/120

14/14 [=====] - 0s 7ms/step - loss: 2.8281 - accuracy: 0.4127 - val_loss: 2.3905 - val_accuracy: 0.4930

Epoch 3/120

14/14 [=====] - 0s 6ms/step - loss: 2.2624 - accuracy: 0.4992 - val_loss: 2.0519 - val_accuracy: 0.5220

Epoch 4/120

14/14 [=====] - 0s 7ms/step - loss: 2.0104 - accuracy: 0.5214 - val_loss: 1.8925 - val_accuracy: 0.5370

Epoch 5/120

14/14 [=====] - 0s 7ms/step - loss: 1.8785 - accuracy: 0.5383 - val_loss: 1.8015 - val_accuracy: 0.5510

Epoch 6/120

14/14 [=====] - 0s 7ms/step - loss: 1.7916 - accuracy: 0.5530 - val_loss: 1.7402 - val_accuracy: 0.5560

Epoch 7/120

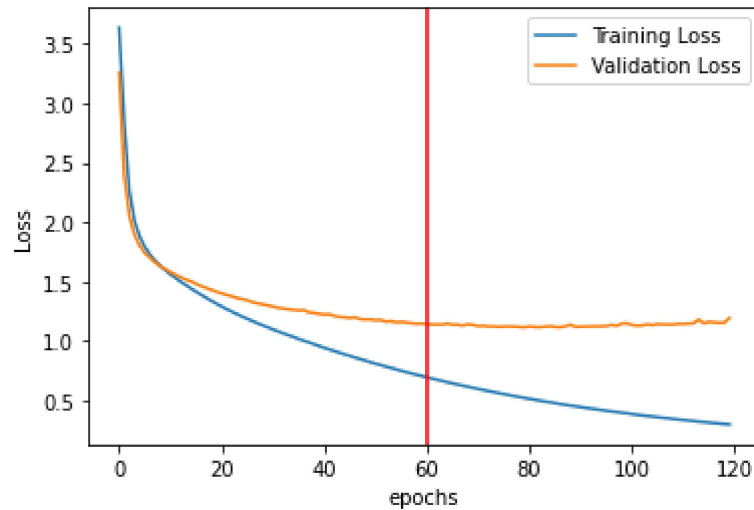
14/14 [=====] - 0s 7ms/step - loss: 1.7367 - accuracy: 0.5654 - val_loss: 1.6887 - val_accuracy: 0.5600

Evaluating

Let's use the plotting functionality covered last week. We will modify it to overlay the validation set loss over the training set on a single figure. Here we have gone back in and used `plt.axvline()` function to plot a vertical line.

```
In [15]: 1 import matplotlib.pyplot as plt
2
3 # If we want to be fancy, we can set a theme by uncommenting the below line.
4 #plt.style.use('ggplot')
5
6 # No subplots here, as we're just drawing over the same plot with multiple data.
7
8 # Loss and Val Loss metrics.
9 loss = model_training_history.history['loss']
10 val_loss = model_training_history.history['val_loss']
11
12
13 # Training Loss, and Validation Loss. Providing labels is useful for our legend later.
14 plt.plot(loss, label="Training Loss")
15 plt.plot(val_loss, label="Validation Loss")
16
17 # X and Y axes labels.
18 plt.ylabel('Loss')
19 plt.xlabel('epochs')
20
21 # Draw a red vertical line at x=60.
22 plt.axvline(x=60, color='red')
23
24 # Display a legend.
25 plt.legend()
```

```
Out[15]: <matplotlib.legend.Legend at 0x7fae12789370>
```



Based on our results here we begin to overfit around the $x=60$ - $x=100$ mark. In the end we have a training accuracy of approx 93% with validation accuracy of 77%.

At this point we should re-train our whole model, stopping at Epoch = 60. Then we can evaluate our model.

Task: Reinitialise model, retrain cutting training off at the optimal point

TODO:

- Add code below to re-create, re-compile, and re-train the NN setting epochs = 40 (or another suitable point where we begin to overfit)
- Add code to generate evaluation metrics for the entire test set. Note, A confusion matrix here would be a 46x46 matrix... Standard accuracy here is fine.

If we predict using the final model after 120 epochs, we've already overfit at this point. We know our model will not generalise well, and we can expect poor test-set performance. The reason for retraining and stopping at the 'sweet spot' is that we end up with a 'good' model which hasn't yet overfit and isn't underfit. It's the best it can currently be (given the architecture so far).

In [16]:

```
1  ### TODO:
2  # Re-create, re-compile, re-train the NN model, setting epochs = 40.
3
4  ### HERE
5
6  pred_y = model.predict(x_test_enc)
7
8  # Let's look at the first prediction output. To get a feeling for the shape, and values.
9  print(pred_y[0].shape)
10 print(pred_y[0])
11 print(np.sum(pred_y[0])) # Softmax output. Should be approx = 1.
12
13 print(np.argmax(pred_y[0])) # Which label is the highest probability
14
15 ### HERE some more comparison code for pred_y against ground truth (y_test) -
16 #NOT our one-hot encoded one. Note how Argmax provides us with the integer representation?
```

```
(46,)
[1.78038806e-03  4.92628850e-03  5.20060817e-03  5.73029229e-03
 8.65137458e-01  9.53347946e-04  3.59962252e-03  6.54906267e-04
 1.34030252e-03  7.57247763e-05  2.84843874e-04  1.18656212e-03
 9.32043826e-04  2.63107615e-03  1.18364044e-03  4.77557862e-03
 4.78170253e-02  1.45531548e-02  1.00136560e-03  1.66241793e-04
 9.55136144e-04  3.53631505e-04  2.80926307e-03  9.32251802e-04
 1.45427068e-03  2.51510763e-03  1.96107969e-04  2.96222558e-03
 8.36023036e-03  1.00723677e-03  1.65941456e-04  2.77865469e-03
 2.75910366e-04  3.99878307e-04  3.00402789e-05  4.65789723e-04
 3.21024272e-04  4.48368490e-04  1.27896760e-03  4.52511944e-03
 1.19660588e-04  1.84255920e-03  5.77301893e-04  1.17847776e-04
 6.99794182e-05  1.10701565e-03]
1.0
4
```

Information Bottleneck

Let's modify our model definition to decrease the number of neurons in the hidden layer. This will behave as a 'pinch' going from 10000 -> 4 -> 4 -> 46. Fundamentally, we're going to observe an inability of our neural network to model the problem's complexity well enough.

In [17]:

```
1 model2 = keras.Sequential()
2 model2.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3 model2.add(layers.Dense(4, activation="relu"))
4 model2.add(layers.Dense(4, activation="relu")) # 2 Hidden Layers
5 model2.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
6 model2.summary()
7
8 model2.compile(
9     loss='categorical_crossentropy',
10    optimizer=SGD(learning_rate=0.05), # Higher Learning rate for demonstration.
11    metrics=['accuracy']
12 )
13
14 # Using train, test, val datasets from before.
15 model2_training_history = model2.fit(
16     x_train_enc_rest,
17     y_train_enc_rest,
18     batch_size = 512,
19     epochs = 120,
20     validation_data = (x_val, y_val)
21 )
22
23 plt.plot(model2_training_history.history['loss'], label="Training Loss")
24 plt.plot(model2_training_history.history['val_loss'], label="Validation Loss")
25 plt.ylabel('Loss')
26 plt.xlabel('epochs')
27 plt.legend()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 4)	40004
dense_4 (Dense)	(None, 4)	20
dense_5 (Dense)	(None, 46)	230
Total params: 40,254		
Trainable params: 40,254		
Non-trainable params: 0		

Epoch 1/120

14/14 [=====] - 0s 13ms/step - loss: 3.7814 - accuracy: 0.2993 - val_loss: 3.7158 - val_accuracy: 0.3650

Epoch 2/120

14/14 [=====] - 0s 7ms/step - loss: 3.6714 - accuracy: 0.3499 - val_loss: 3.6058 - val_accuracy: 0.3650

An overall accuracy of approx 62%. Validation accuracy of approx 60%. The overall loss is far from 0 indicating more training to be done; however, the validation loss has begun to increase indicating overfitting. This is an excellent candidate for adding more neurons / hidden layers. Once we get the training accuracy up, we can also then look at regularisation to address the overfitting.

Dropout

Dropout: alpha = 0.2

In our first example, we might want to try using Dropout to help reduce the gap between validation loss curves and training loss curves. Let's set that up.

We can add Dropout using `layers.Dropout()` layer in our Sequential model, remembering to provide a percentage dropout. Usually 0.2-0.5 are good values.

Here we're going to train for a bit longer, as we would expect Dropout to increase training times slightly.

In [18]:

```
1 model3 = keras.Sequential()
2 model3.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3
4 model3.add(layers.Dense(64, activation="relu"))
5 model3.add(layers.Dropout(0.2))
6
7 model3.add(layers.Dense(64, activation="relu")) # 2 Hidden Layers
8 model3.add(layers.Dropout(0.2))
9
10 model3.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
11 model3.summary()
12
13 model3.compile(
14     loss='categorical_crossentropy',
15     optimizer=SGD(learning_rate=0.05), # Higher learning rate for demonstration.
16     metrics=['accuracy']
17 )
18
19 # Using train, test, val datasets from before.
20 model3_training_history = model3.fit(
21     x_train_enc_rest,
22     y_train_enc_rest,
23     batch_size = 512,
24     epochs = 240, # train for a bit longer to get a better picture.
25     validation_data = (x_val, y_val)
26 )
```

Epoch 1/240															
11/11	5	1	0	13	6	1	0	6633	0	3374	1	1	0	3347	1

In [19]:

```
1 model4 = keras.Sequential()
2 model4.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3
4 model4.add(layers.Dense(64, activation="relu"))
5 model4.add(layers.Dropout(0.5))
6
7 model4.add(layers.Dense(64, activation="relu")) # 2 Hidden Layers
8 model4.add(layers.Dropout(0.5))
9
10 model4.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
11 model4.summary()
12
13 model4.compile(
14     loss='categorical_crossentropy',
15     optimizer=SGD(learning_rate=0.05), # Higher learning rate for demonstration.
16     metrics=['accuracy']
17 )
18
19 # Using train, test, val datasets from before.
20 model4_training_history = model4.fit(
21     x_train_enc_rest,
22     y_train_enc_rest,
23     batch_size = 512,
24     epochs = 240, # train for a bit longer to get a better picture.
25     validation_data = (x_val, y_val)
26 )
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
=====	=====	=====
dense_9 (Dense)	(None, 64)	640064
dropout_2 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 64)	4160
dropout_3 (Dropout)	(None, 64)	0
dense_11 (Dense)	(None, 46)	2990
=====	=====	=====
Total params: 647,214		
Trainable params: 647,214		
Non-trainable params: 0		
=====		
Epoch 1/240		
11/1115		

Evaluative graphs

Loss

We're going to overlay our original networks Training loss, and Validation Loss on this graph to see how we compare in an absolute way. Remember, the closer the loss is to 0, theoretically the better our network.

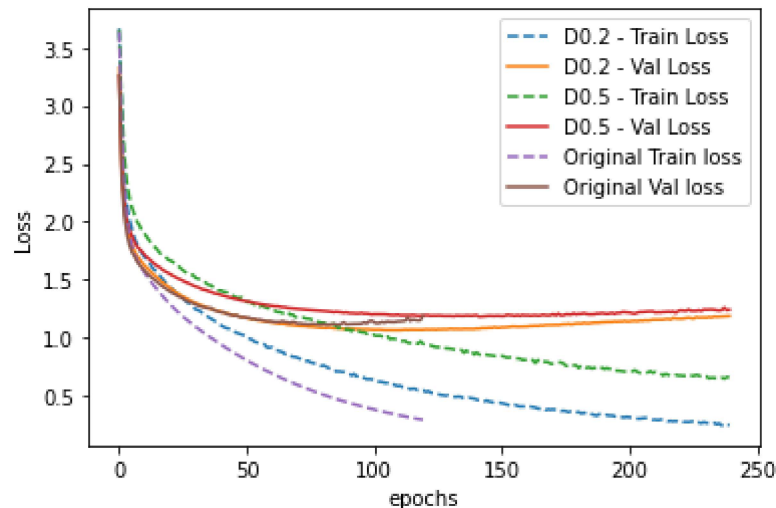
To differentiate between Training loss and Validation Loss, we've used dashed lines for training, and solid for validation. Specifically here you should focus on how low the overall loss goes, the divergence between architectures of their training and validation loss curves, and the moment of inflection of the validation loss curves.

```

In [20]: 1 plt.plot(model3_training_history.history['loss'], '--', label="D0.2 - Train Loss")
          2 plt.plot(model3_training_history.history['val_loss'], label="D0.2 - Val Loss")
          3
          4 plt.plot(model4_training_history.history['loss'], '--', label="D0.5 - Train Loss")
          5 plt.plot(model4_training_history.history['val_loss'], label="D0.5 - Val Loss")
          6
          7 plt.plot(model_training_history.history['loss'], '--', label="Original Train loss")
          8 plt.plot(model_training_history.history['val_loss'], label="Original Val loss")
          9
         10 plt.ylabel('Loss')
         11 plt.xlabel('epochs')
         12 plt.legend()

```

Out[20]: <matplotlib.legend.Legend at 0x7faae1e20dc0>



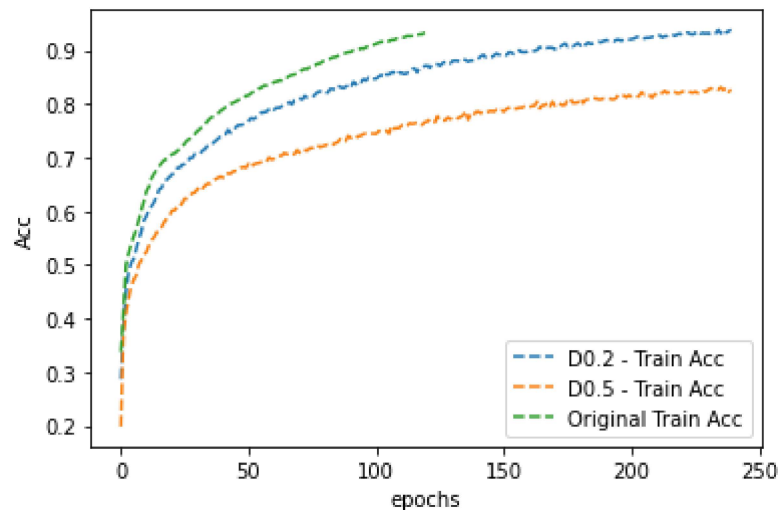
Accuracy

Looking at the accuracy graphs we can see that the original training accuracy curve improves faster than the others. Intuitively this makes sense, as we have introduced a form of regularisation, we may take longer to get to the 'end' of our training.

With Dropout, looking at our accuracy can also be misleading. During training time, we are dropping 20-50% of the neuron connections; however, at test time we are keeping them all.

```
In [21]: 1 plt.plot(model3_training_history.history['accuracy'], '--', label="D0.2 - Train Acc")
2 plt.plot(model4_training_history.history['accuracy'], '--', label="D0.5 - Train Acc")
3 plt.plot(model_training_history.history['accuracy'], '--', label="Original Train Acc")
4
5 plt.ylabel('Acc')
6 plt.xlabel('epochs')
7 plt.legend()
```

Out[21]: <matplotlib.legend.Legend at 0x7faae1df9340>



Task: Plot Testing Accuracy

Using the three previously trained models, plot the testing accuracy on a chart. Does this fall in line with our expectations? How has the regularisation helped?

Batch Normalisation

In our reading we introduced Batch Normalisation as a means to ensure that the variances from earlier layers do not drift too far as we propagate through our network. It also benefits us during training by reducing covariance shift; ensuring that gradient application doesn't then impact other layers too drastically.

For the application of this layer, we must separate out our non-linearities. We insert the BN between the activation function, and the neurons. Dense
-> BN -> Activation. As per the original paper; however, recent code created by these authors shows BN applied after activation (ReLU) - this is still

In [22]:

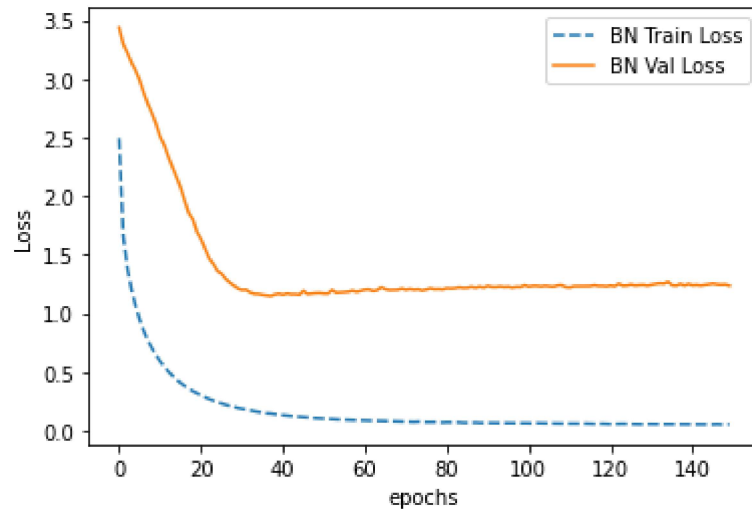
```
1 bn_model = keras.Sequential()
2 bn_model.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3
4 bn_model.add(layers.Dense(64))
5 bn_model.add(layers.BatchNormalization())
6 bn_model.add(layers.ReLU())
7
8 bn_model.add(layers.Dense(64))# 2 Hidden Layers
9 bn_model.add(layers.BatchNormalization())
10 bn_model.add(layers.ReLU())
11
12 bn_model.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
13 bn_model.summary()
14
15 bn_model.compile(
16     loss='categorical_crossentropy',
17     optimizer=SGD(learning_rate=0.05), # Higher Learning rate for demonstration.
18     metrics=['accuracy']
19 )
20
21 # Using train, test, val datasets from before.
22 bn_model_training_history = bn_model.fit(
23     x_train_enc_rest,
24     y_train_enc_rest,
25     batch_size = 512,
26     epochs = 150, # train for a bit longer to get a better picture.
27     validation_data = (x_val, y_val)
28 )
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
=====		
dense_12 (Dense)	(None, 64)	640064
batch_normalization (Batch Normalization)	(None, 64)	256
re_lu (ReLU)	(None, 64)	0
dense_13 (Dense)	(None, 64)	4160
batch_normalization_1 (Batch Normalization)	(None, 64)	256
re_lu_1 (ReLU)	(None, 64)	0
dense_14 (Dense)	(None, 46)	2990
=====		
Total params: 647,726		
Total params: 647,726		

```
In [23]: 1 plt.plot(bn_model_training_history.history['loss'], '--', label="BN Train Loss")
2         plt.plot(bn_model_training_history.history['val_loss'], label="BN Val Loss")
3
4         plt.ylabel('Loss')
5         plt.xlabel('epochs')
6         plt.legend()
```

Out[23]: <matplotlib.legend.Legend at 0x7facfc186400>



Obseervations

This training graph appears to improve training loss incredibly quickly, at a rate higher than most we've seen so far. However, alongside this, we notice that the validation loss has diverged significantly since around epoch 20, and is slowly going back up.

We would need to investigate the train / val splits to see if it's a data partitioning issue, then look at ways to add additional regularisation, or perhaps tweak model complexity.

L1 / L2 Regularisation

In our reading we introduced a way to prevent weights from changing too much, or too drastically. Primarily we can look at doing this via L1, L2, or a mix of the two (L1_L2). This can be done for the weights, as well as the biases of a layer (and even to the activation output!).

Let's look at if this can help us with the above.

In [24]:

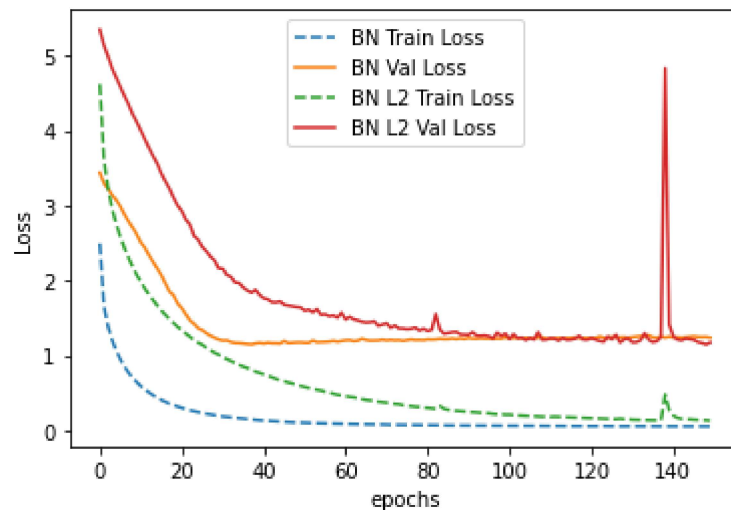
```
1 bn_l2_model = keras.Sequential()
2 bn_l2_model.add(layers.InputLayer(input_shape=(10000,))) # 10000 dimensional input.
3
4 bn_l2_model.add(layers.Dense(64, kernel_regularizer='l2'))
5 bn_l2_model.add(layers.BatchNormalization())
6 bn_l2_model.add(layers.ReLU())
7
8 bn_l2_model.add(layers.Dense(64, kernel_regularizer='l2'))# 2 Hidden Layers
9 bn_l2_model.add(layers.BatchNormalization())
10 bn_l2_model.add(layers.ReLU())
11
12 bn_l2_model.add(layers.Dense(46, activation="softmax")) # 46 Output topics possible.
13 bn_l2_model.summary()
14
15 bn_l2_model.compile(
16     loss='categorical_crossentropy',
17     optimizer=SGD(learning_rate=0.05), # Higher Learning rate for demonstration.
18     metrics=['accuracy']
19 )
20
21 # Using train, test, val datasets from before.
22 bn_l2_model_training_history = bn_l2_model.fit(
23     x_train_enc_rest,
24     y_train_enc_rest,
25     batch_size = 512,
26     epochs = 150, # train for a bit longer to get a better picture.
27     validation_data = (x_val, y_val)
28 )
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
=====		
dense_15 (Dense)	(None, 64)	640064
batch_normalization_2 (Batch Normalization)	(None, 64)	256
re_lu_2 (ReLU)	(None, 64)	0
dense_16 (Dense)	(None, 64)	4160
batch_normalization_3 (Batch Normalization)	(None, 64)	256
re_lu_3 (ReLU)	(None, 64)	0
dense_17 (Dense)	(None, 46)	2990
=====		
Total params: 647,726		
Total size: 647,726 bytes		

```
In [25]: 1 plt.plot(bn_model_training_history.history['loss'], '--', label="BN Train Loss")
2 plt.plot(bn_model_training_history.history['val_loss'], label="BN Val Loss")
3
4 plt.plot(bn_l2_model_training_history.history['loss'], '--', label="BN L2 Train Loss")
5 plt.plot(bn_l2_model_training_history.history['val_loss'], label="BN L2 Val Loss")
6
7 plt.ylabel('Loss')
8 plt.xlabel('epochs')
9 plt.legend()
```

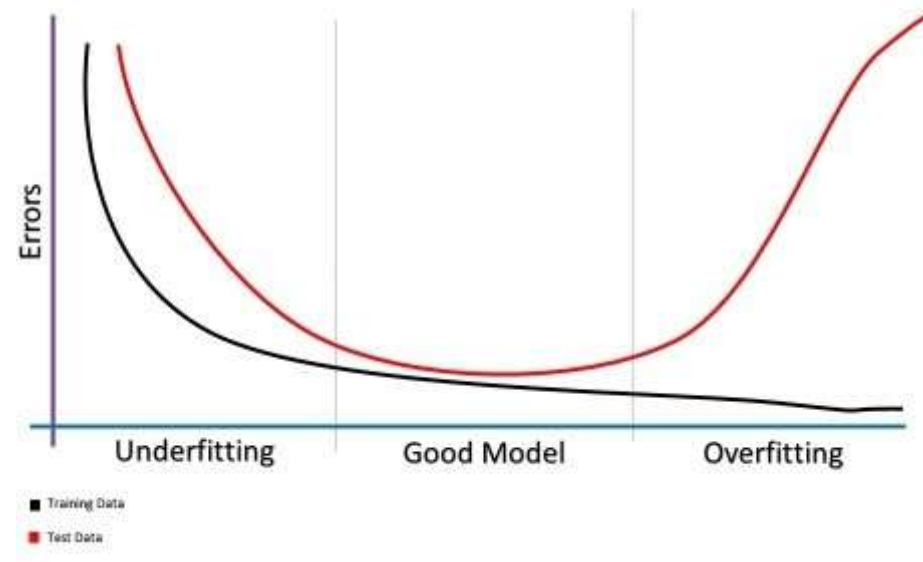
Out[25]: <matplotlib.legend.Legend at 0x7faadd255820>



Observations

The L2 regularised approach seems to take longer to get to the plateau, but generally results in the validation loss following the shape of the training loss. The only difference now is the large gap between them, which could be explained by dataset split differences. Our training accuracy is high, at approx 97%, and the validation loss appears to just about be flattening. We may want to look at training for more epochs to confirm a good stopping point.

Final Remarks



Source: <https://www.baeldung.com/cs/learning-curve-ml> (<https://www.baeldung.com/cs/learning-curve-ml>)

Training Loss Curve not decreasing, or Training loss stagnates

This could indicate your model does not have sufficient capacity

Train and Validation Loss Diverging

If your validation loss diverges from your training loss, with your validation loss flatlining or even going up again then your model is overfit. Potential solutions:

- Add regularisation (Dropout / BN / L1 / L2 / etc)
- Reduce Model Capacity (if model is overly complex)
- Verify your Train and Validation splits are statistically comparable (Try change the shuffle seed.). Divergence could be caused by the sets being different.

